深度优先搜索

depth-first search，DFS

深度优先探索，深度优先搜索

depth-first exploration

对一个有向图的**深度优先探索**有规律地访问它的顶点和走过它的边。走过一条边分为两个阶段，先是在这条边上**前进**，以后还要在这条边上**回退**。

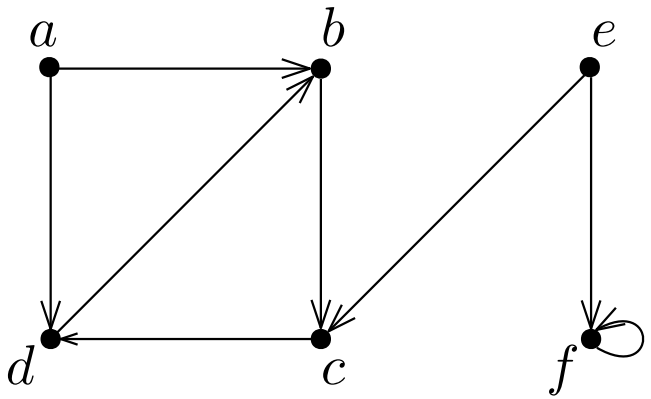
advance retreat

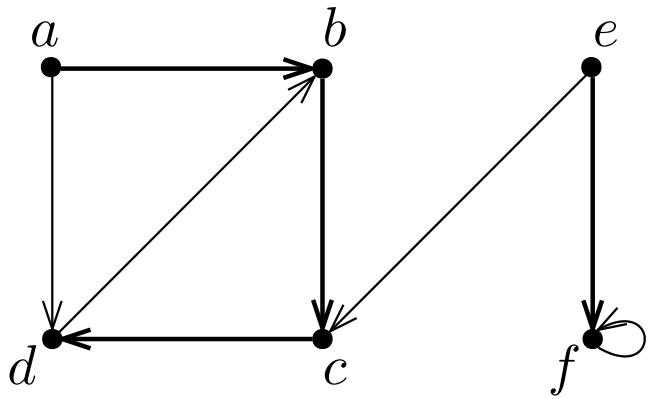
current vertex

维护一个**当前点** v，最初是 null。把所有顶点标记为未访问过，所有边标记为未走过。

重复下列三种操作之一直到所有点都访问过，所有边都走过：

1. 当前点 v 是 null 且有顶点未访问过。令 v 是任一未访问过的点。**访问** v。这开启一次以 v 为起点的**深度优先搜索**（depth-first search），将访问所有从 v 可达的未访问过的顶点，走过所有从这些点发出的边。
2. 当前点不是 null 且有未走过的出边。任选一条这样的边 e，不妨设是从 v 到 w 的，沿着这条边**前进**。若 w 已访问过，立即在 e 上**回退**；否则以 e 为进入点 w 的**树边**，把当前点 v 置为 w，**访问**这新的 v。
3. 当前点 v 不是 null 且它没有没走过的出边。若 v 是此次 DFS 的起点，将当前点 v 置为 null。否则有一条树边 e 进入点 v，设 e 的起点是 u；在 e 上**回退**，把当前点 v 置为 u。





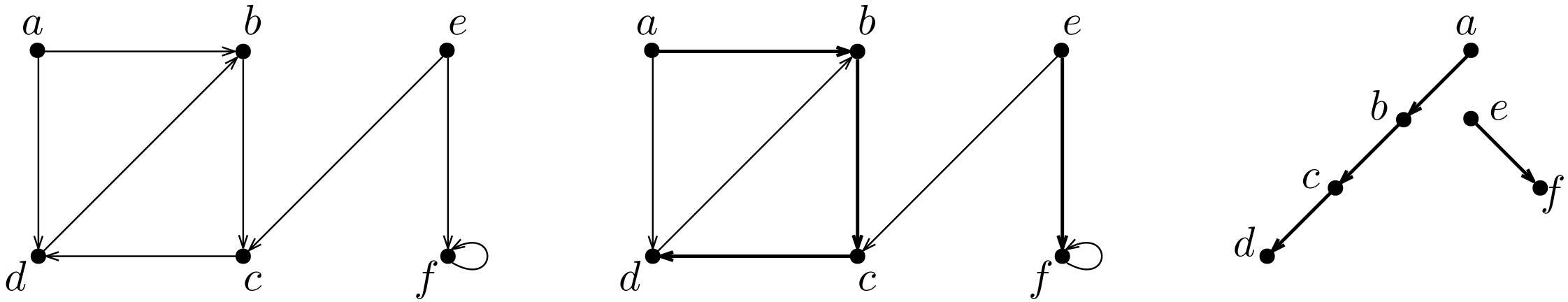
引理 D1　一次深度优先探索产生一族**有根树**，以每次 DFS 的起点为根，它们的边是此次探索定义的树边。每个树的顶点是在起于它的根的那次 DFS 期间被访问的那些点。

证明：

- 每个点从未访问变成已访问只有一次，所以至多有一条进入它的树边。
- 每条新的树边都进入一个未访问过的点，此时还没有从它发出的树边，所以树边不成环。
- 每个点或者是一次 DFS 的起点，那样就没有树边进入它而它是一个树的根；或者不是起点，那样就有树边进入它而它不是根。
- 若 e 是从 v 到 w 的树边，则访问了点 v 的那次 DFS 也访问了点 w。

DFS 树，DFS 森林

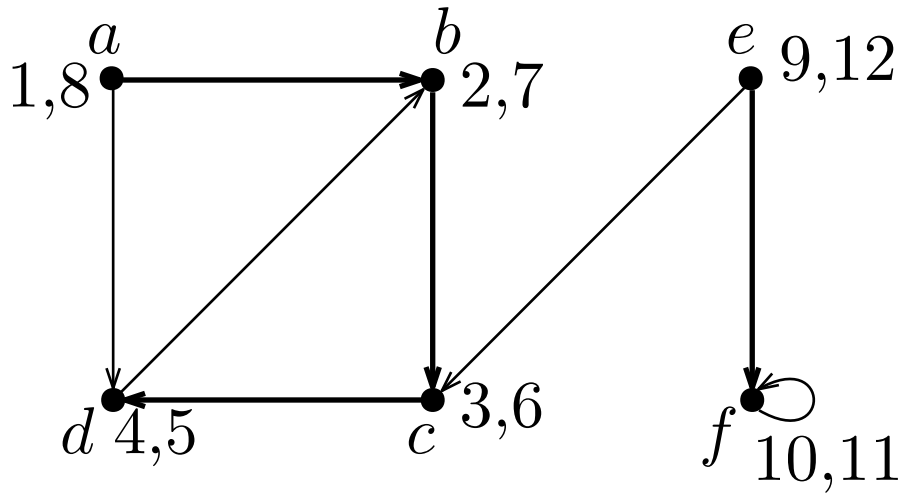
我们把一次深度优先探索产生的那些有根树称为 DFS 树。它们一起构成这次探索产生的 DFS 森林。



先序访问，后序访问，时间戳

深度优先探索以先序访问每个 DFS 树的顶点。我们称这些访问为**先序访问**（preorder visit）或**初访**。此外，我们还考虑深度优先探索对每个顶点 v 的第二次隐含的访问，就是上述情况 3，当 v 是当前点但它没有未走过的出边时。我们把这些访问称为**后序访问**（postorder visit）或**终访**，因为在每个 DFS 树上，这些访问发生的顺序是后序。

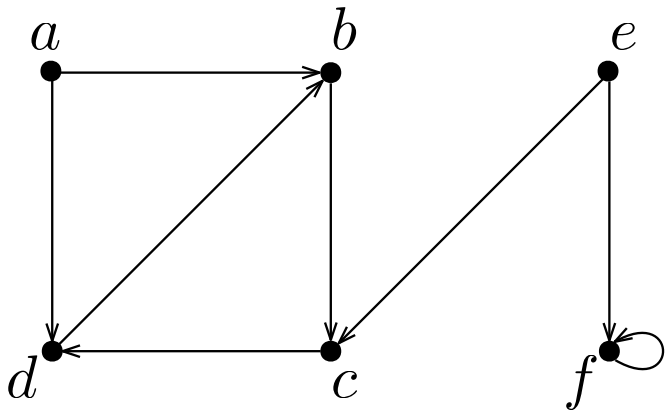
我们给这 $2n$ 次初访和终访每个赋予一个数，称为**时间戳**。我们把点 v 的初访和终访的时间戳分别记作 $v.pre$ 和 $v.post$。这些时间戳的值不重要，只要能反映顺序：若一次访问发生在另一次访问之前，则前者的时间戳必小于后者。

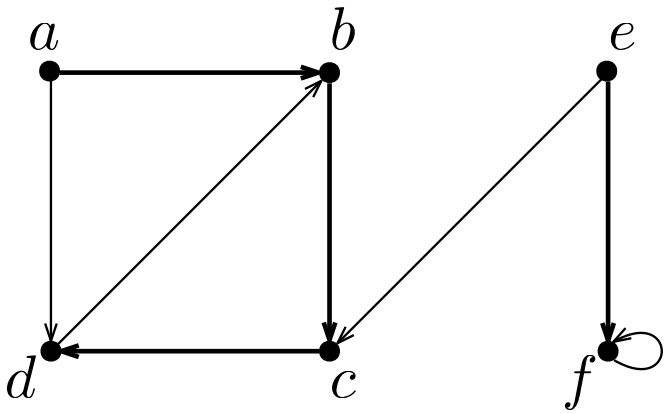


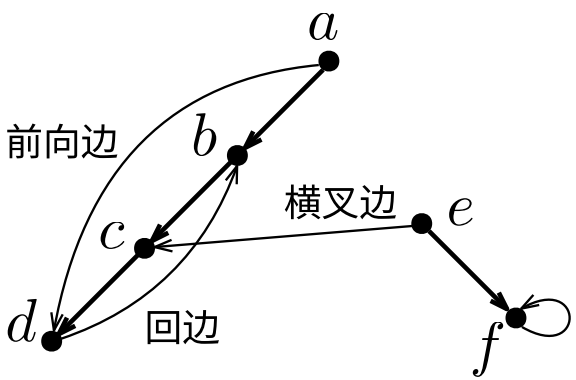
回边，前向边，横叉边

一次深度优先探索把除树边和自环之外的边 (v, w) 分成三类。

- **回边**：w 是 v 的祖先。也称返祖边。
- **前向边**：w 是 v 的后代。
- **横叉边**：v 和 w 无祖先——后代关系。

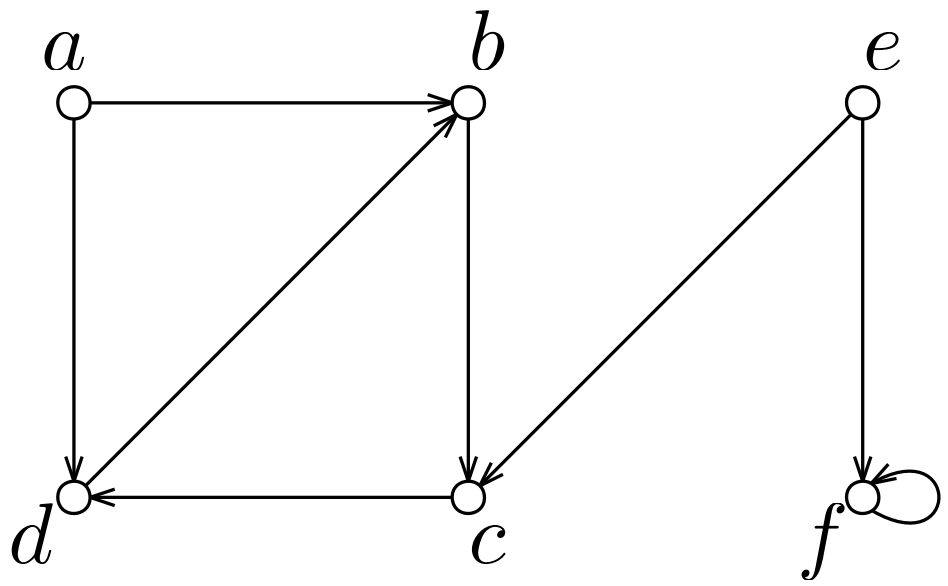


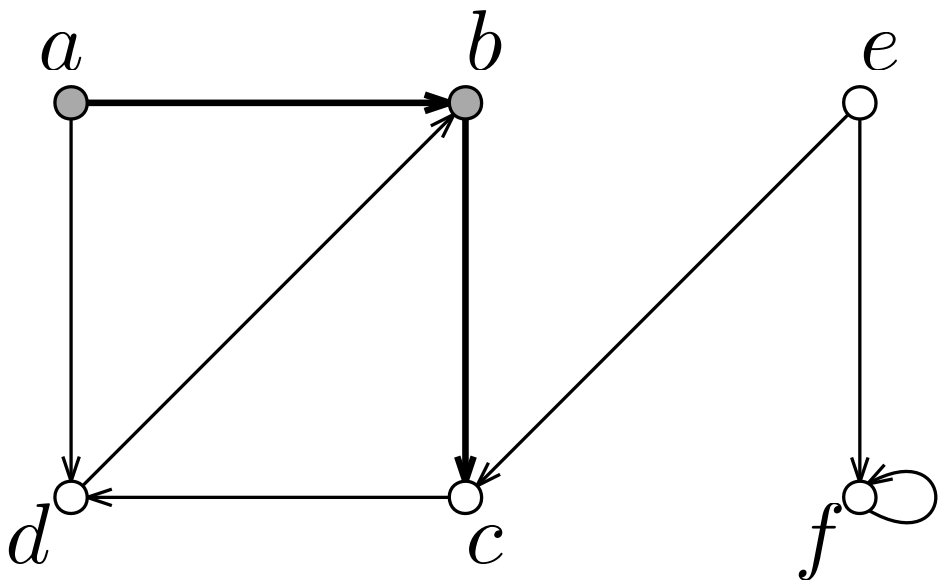




当前路径

DFS 维护一条从起点到当前点，由树边构成的路径。在一条树边上前进使此路径多一条边，而在一条树边上后退使它少一条边。我们称此路径为**当前路径**。





引理 D2 每个顶点 v 在当前路径上是从它刚要被初访时开始直到它刚被终访过为止。在这段时间内，DFS 在 v 的每条出边上前进和回退。

证明： 一个点 v 被初访是在它变成当前点并加入当前路径时。自此它在当前路径上直到被终访。如果在 v 的某条出边上的前进还没发生，点 v 的终访不可能发生。在 v 的出边上前进时，v 是当前点。假设在边 $e = (v, w)$ 上的前进刚刚发生。若 w 访问过，在边 e 上的回退随即发生。否则 e 成为树边而 w 变成当前点。在点 v 再次成为当前点之前 v 不会被终访，而那只能发生在 w 被终访以后。在 w 被终访之后，在 e 上的回退立即发生，故而是在 v 被终访之前。

引理 D3　下列四个条件等价：

- (1) 顶点 w 是顶点 v 的后代
- (2) $v.pre \leq w.pre < v.post$
- (3) $v.pre \leq w.post \leq v.post$
- (4) $v.pre \leq w.pre < w.post \leq v.post$

证明：若 w 是 v 的后代，则 w 被加入和删除当前路径时 v 都在当前路径上。根据引理 D2，$v.pre \leq w.pre < w.post \leq v.post$。因此 (1) 蕴含 (2)，(3)，(4)。

若 $v.pre \leq w.pre < v.post$，则 w 被初访时 v 在当前路径上，故 w 是 v 的后代；即 (2) 蕴含 (1)。类似地，若 $v.pre < w.post \leq v.post$，$w$ 被终访时 v 在当前路径上，故 w 是 v 的后代；即 (3) 蕴含 (1)。最后，(4) 蕴含 (2) 和 (3) 故也蕴含 (1)。

引理 D4　点 w 是点 v 的后代当且仅当在 v 刚要被初访时有一条从 v 到 w 的路径，上面都是未访问过的点。

证明：

- 若 w 是 v 的后代，根据引理 D3，在 v 刚要被初访时，所有从 v 走树边到 w 的路径上的点都未访问。
- 反之，设在 v 刚要被初访时有一条从 v 到 w 的路径 P，上面的点都未访问。我们证明 w 是 v 的后代。
  - 若边 (x, y) 在 P 上而 x 是 v 的后代，则据引理 D2 有 $y.pre < x.post$，据引理 D3 又有 $x.post \leq v.post$，故 $y.pre < v.post$。又根据条件，v 刚要被初访时 y 尚未被访问，故 $v.pre < y.pre$。综上，$v.pre < y.pre < v.post$，所以 y 是 v 的后代。
  - 于是可对路径 P 的顶点数用**归纳法**，因为在 P 上只有一个点 v 时，v 是自身的后代，结论成立。

引理 D5　设 v 是顶点，e 是从 x 到 y 的边。
(1) 若 x 是 v 的后代而 y 不是，则 $y.pre < v.pre$。
(2) 若 y 是 x 的后代而 x 不是，则 $v.post < x.post$。

证明：此引理是引理 D3 的推论。

(1) 设 x 是 v 的后代且 $v.pre \leq y.pre$。由于 x 是 v 的后代，在边 e 上从 x 前进到 y 发生在 v 的终访之前，故而 y 的初访也发生在 v 的终访之前。即有 $y.pre < v.post$。根据引理 D3，此不等式和 $v.pre \leq y.pre$ 蕴含 y 是 v 的后代。

(2) 设 y 是 v 的后代且 $x.post \leq v.post$。由于在边 e 上回退是在 x 被终访之前，有 $y.pre < x.post \leq v.post$。由于 y 是 v 的后代，有 $v.pre \leq y.pre$。这些不等式合起来给出 $v.pre < x.post \leq v.post$。据引理 D3，$x$ 是 v 的后代。

引理 D6　若有从 x 到 y 的横叉边，则 $y.post < x.pre$。

证明：由于 y 不是 x 的后代，以 $v = x$ 代入引理 D5 (1) 给出 $y.pre < x.pre$。而根据引理 D3，$y.pre < x.pre$ 和 $x.pre < y.post$ 蕴含 y 是 x 的后代，矛盾。故而 $y.post < x.pre$。

DFS 的递归实现

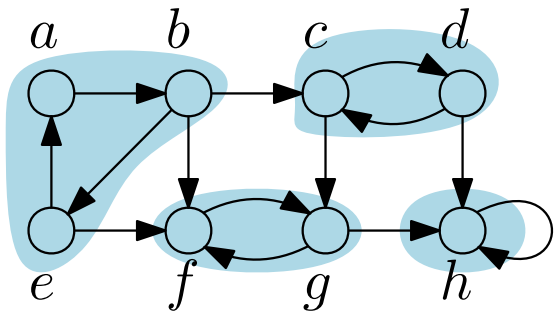
```
dfe(V):
    start(V)
    for s ∈ V :
        if unvisited(s) :
            dfs(s)

dfs(v):
    previsit(v)
    scan(v)
    postvisit(v)

scan(v):
    for a ∈ out(v) :
        w ← a.tip
        advance(v, a, w)
        if unvisited(w) :
            tree-advance(v, a, w)
            dfs(w)
            tree-retreat(v, a, w)
        else:
            nontree-traverse(v, a, w)
        retreat(v, a, w)

start(V):
    time ← 0
    for v ∈ V :
        v.pre ← 0

unvisited(v):
    return v.pre = 0

previsit(v):
    time ← time + 1
    v.pre ← time

postvisit(v):
    time ← time + 1
    v.post ← time
```

强连通分量

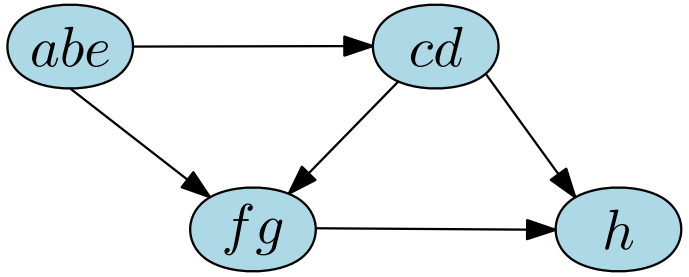
在有向图 G 上，称两顶点 u, v **相互可达**，若有从 u 到 v 路径和从 v 到 u 的路径。相互可达是等价关系，它把顶点集分成等价类，称为 G 的**强连通分量**（strongly connected component）或**强分量**（strong component）。只有一个强连通分量的有向图称为**强连通的**（strongly connected）。

我们常把强连通分量简写为 SCC。

若把 G 的每个 SCC 看成一个点，就得到 G 的 SCC 图，它是一个 dag。



G



G^{SCC}

强连通分量的性质

引理 S1 若 u 和 v 在同一个 SCC S 里，则任一从 u 到 v 的路径上的顶点全在 S 里。

证明： 若 u 和 v 在 S 里，则有一条从 v 到 u 的路径 P_1。若 x 在从 u 到 v 的路径 P_2 上，则 x 在由 P_1, P_2 构成的环上，因此 x 在 S 里。

引理 S2　在一个 SCC S 里，$v.pre$ 最小的点 v 也是 $v.post$ 最大的点，这个点是 S 里每个点的祖先。

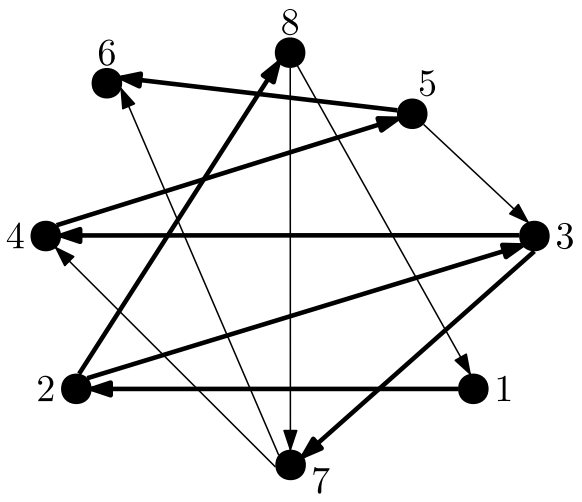
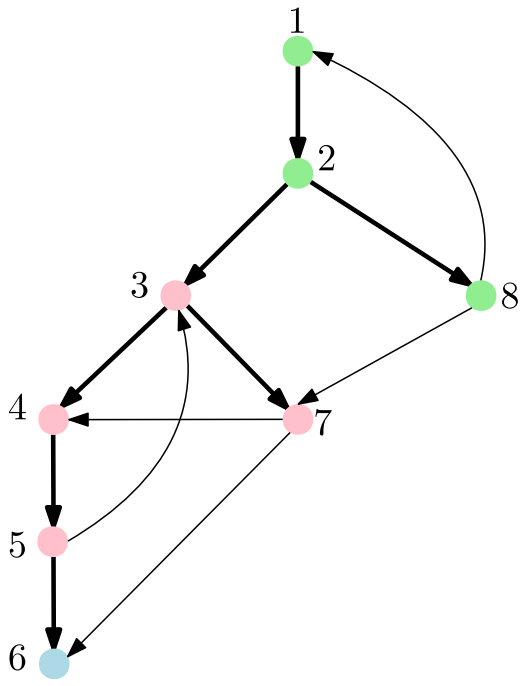
证明：此引理是引理 D4 的推论。S 里 $v.pre$ 最小的点 v 是 S 里第一个被访问的点。根据引理 D4，v 是 S 里所有点的祖先。根据引理 D3，v 也是 S 里 $v.post$ 最大的点。

首领，随从

我们把一个 SCC 里 $v.pre$ 最小的点称为**首领**，而其余点称为首领的**随从**。

每个 DFS 树的根一定是它所属的 SCC 的首领，但是首领未必是 DFS 树的根。

引理 S3 若 y 不是它所属的 SCC S 的首领，则 y 的父节点 x 也在 S 里。

证明： 设 $v \neq y$ 是 S 的首领。树边 (x, y) 和一条从 y 到 v 的路径构成一条从 x 到 v 的路径。根据引理 S2，y 是首领 v 的后代，所以从 v 走树边可以到 y，而且会经过 x。所以 x 在 S 里。

引理 S3